

TEORIA.PDF

Documento de Respostas Teóricas — Ficha de Exercícios I

Elementos do Grupo
Américo Penha
Joel Ali Viano Pinto
João Filipe João
Kelven Jossamo
Letícia Figueira Fam

1. Structs em C

Exercício 1 — struct Aluno com variável e valores

Uma **struct** é um tipo de dado composto que agrupa variáveis de diferentes tipos sob um único nome. Criamos a struct *Aluno* com os campos **nome** (cadeia de caracteres), **idade** (inteiro) e **media** (número real).

```
struct Aluno { char nome[50]; int idade; float media; };  
struct Aluno a1;  
strcpy(a1.nome, "Maria Joao"); a1.idade = 20; a1.media = 15.5;
```

Exercício 2 — typedef com 3 variáveis

O **typedef** cria um alias para um tipo, eliminando a necessidade de escrever *struct* em cada declaração. Com `typedef struct { ... } Aluno;` podemos declarar variáveis simplesmente como *Aluno a1, a2, a3;*.

Exercício 3 — Leitura e impressão de 5 alunos

Declara-se um vetor *Aluno turma[5]*; e usa-se um ciclo **for** com *scanf()* para preencher os campos de cada aluno. A impressão é feita com *printf()* acedendo a cada campo via operador ponto (.).

Exercício 4 — Função estaAprovado()

A função recebe uma struct *Aluno* por valor e retorna 1 (aprovado) se a média for maior ou igual a 10, ou 0 (reprovado) caso contrário.

```
int estaAprovado(Aluno a) { return a.media >= 10.0; }
```

Exercício 5 — Vetor de Produto e valor total do estoque

Declara-se um vetor de *struct Produto* com campos **nome**, **preco** e **quantidade**. O valor total é calculado acumulando $preco \times quantidade$ de cada produto num ciclo.

Exercício 6 — Ordenação de Alunos por média

Usa-se o algoritmo **Bubble Sort** comparando o campo *media* de structs adjacentes. Quando $turma[j].media > turma[j+1].media$, as structs completas são trocadas usando uma variável temporária do tipo Aluno.

2. Algoritmos de Busca

Exercício 7 — Busca Linear

A **busca linear** percorre o vetor elemento a elemento, da posição 0 até $n-1$, comparando cada elemento com o valor procurado. Retorna o índice se encontrar, ou -1 se o elemento não existir. Funciona em vetores ordenados e não ordenados.

```
for (int i = 0; i < n; i++) {
    if (arr[i] == alvo) return i;
}
return -1;
```

Exercício 8 — Busca Linear com contagem de comparações

Adicionamos um ponteiro `int *comparacoes` como parâmetro. A cada iteração do ciclo, incrementamos o contador `(*comparacoes)++` antes da comparação, permitindo medir o custo real da busca.

Exercício 9 — Busca Binária Iterativa

A **busca binária** exige vetor **ordenado**. A cada passo, calcula o elemento do meio ($meio = esq + (dir-esq)/2$) e descarta metade do vetor conforme o alvo seja maior ou menor que o elemento do meio.

```
int esq=0, dir=n-1;
while (esq <= dir) {
    int meio = esq + (dir-esq)/2;
    if (arr[meio]==alvo) return meio;
    else if (arr[meio]<alvo) esq=meio+1;
    else dir=meio-1;
}
```

Exercício 10 — Busca Binária Recursiva

A versão recursiva substitui o ciclo por chamadas recursivas. O caso base é $esq > dir$ (elemento não encontrado). A cada chamada, o intervalo de pesquisa é reduzido a metade, resultando em profundidade máxima de $O(\log n)$.

```
int buscaRec(int arr[], int esq, int dir, int alvo) {
    if (esq > dir) return -1;
    int meio = esq + (dir-esq)/2;
    if (arr[meio]==alvo) return meio;
    if (arr[meio]<alvo) return buscaRec(arr,meio+1,dir,alvo);
    return buscaRec(arr,esq,meio-1,alvo);
}
```

Exercício 11 — Por que a busca binária não se aplica a vetores não ordenados?

A busca binária baseia-se na propriedade de **ordem**: ao comparar o elemento do meio com o alvo, é possível determinar com certeza em qual metade o alvo se encontra — porque todos os elementos à esquerda são menores e todos à direita são maiores.

Num vetor **não ordenado** esta propriedade não existe. Ao descartar metade do vetor, podemos estar a eliminar justamente a metade que contém o elemento procurado, levando a resultados incorretos.

Exemplo: vetor {42, 5, 23, 8, 16, 90, 3} — o valor 42 está no índice 0, mas a busca binária retorna -1 (não encontrado) porque descarta a metade errada.

Exercício 12 — Comparação experimental: Linear vs Binária

Configuração	Alvo	Busca Linear	Busca Binária
Vetor pequeno (10 elem.)	19 (último)	10 comparações	4 comparações
Vetor grande (1000 elem.)	1999 (último)	1000 comparações	10 comparações

Conclusão: para 1000 elementos, a busca binária necessitou apenas 10 comparações contra 1000 da busca linear — 100× mais eficiente no pior caso.

3. Análise de Complexidade

Exercício 13 — Complexidade dos trechos de código

a) Loop simples

```
for (i=0; i<n; i++) { printf("%d", i); }
```

Complexidade: $O(n)$ — o ciclo executa exactamente n iterações, uma por cada valor de i de 0 a $n-1$.

b) Dois loops aninhados

```
for (i=0; i<n; i++)  
    for (j=0; j<n; j++) { printf("%d", i+j); }
```

Complexidade: $O(n^2)$ — para cada valor de i (n vezes), o ciclo interno executa n vezes. Total: $n \times n = n^2$ operações.

c) Loop com crescimento exponencial

```
for (i=1; i<n; i*=2) { printf("%d", i); }
```

Complexidade: $O(\log n)$ — i dobra a cada iteração ($1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow \dots$). O número de iterações é $\log_2(n)$. Exemplo: para $n=64$, são apenas 6 iterações.

Trecho	Complexidade	Razão
a) Loop simples	$O(n)$	n iterações
b) Loops aninhados	$O(n^2)$	$n \times n$ iterações
c) Loop com $i*=2$	$O(\log n)$	i dobra a cada passo

Exercício 16 — Classificação de casos: Busca Linear e Binária

Algoritmo	Melhor Caso	Caso Médio	Pior Caso
Busca Linear	$O(1)$	$O(n)$	$O(n)$
Busca Binária	$O(1)$	$O(\log n)$	$O(\log n)$

- **Busca Linear — Melhor caso $O(1)$:** o elemento procurado está na 1.^a posição.
- **Busca Linear — Pior caso $O(n)$:** o elemento está na última posição ou não existe.
- **Busca Binária — Melhor caso $O(1)$:** o elemento está exactamente no meio do vetor.
- **Busca Binária — Pior caso $O(\log n)$:** o elemento está no extremo ou não existe.

4. Ordenação (Sorting)

Exercício 17 — Bubble Sort

O **Bubble Sort** percorre repetidamente o vetor comparando pares de elementos adjacentes e trocando-os se estiverem fora de ordem. A cada passagem completa, o maior elemento não ordenado 'sobe' para a sua posição final (como uma bolha).

```
for (i=0; i<n-1; i++)
    for (j=0; j<n-i-1; j++)
        if (arr[j] > arr[j+1]) { troca(arr[j], arr[j+1]); }
```

Exercício 18 — Bubble Sort Otimizado

A versão otimizada introduz uma **flag** booleana *trocou*. Se numa passagem completa nenhuma troca for realizada, o vetor já está ordenado e o algoritmo termina antecipadamente, evitando iterações desnecessárias.

Ganho no melhor caso: vetor já ordenado → apenas 1 passagem → $O(n)$.

Exercício 19 — Comparação de complexidade dos algoritmos de ordenação

Algoritmo	Melhor Caso	Caso Médio	Pior Caso	Estável	Espaço
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Sim	$O(1)$
Bubble Sort Otimizado	$O(n)$	$O(n^2)$	$O(n^2)$	Sim	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Não	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sim	$O(1)$

Observações:

- Todos os três algoritmos têm complexidade $O(n^2)$ no pior e médio caso.
- **Insertion Sort** e **Bubble Sort Otimizado** atingem $O(n)$ no melhor caso (vetor já ordenado), sendo mais eficientes para dados quase ordenados.
- **Selection Sort** realiza sempre $O(n^2)$ comparações, independentemente da ordem inicial — no entanto, faz no máximo $O(n)$ trocas.
- Todos são algoritmos **in-place** (espaço auxiliar $O(1)$).

Exercício 20 — Comparação de trocas: Bubble Sort vs Selection Sort

O programa gera um vetor aleatório e ordena-o com os dois algoritmos contando o número de trocas. O Selection Sort realiza sempre exatamente $n-1$ trocas no máximo (uma por posição), enquanto o Bubble Sort pode realizar muito mais trocas, pois troca elementos em cada comparação.

5. Sistema de Gestão de Alunos

Exercício 21 — Sistema completo com struct

O sistema utiliza um vetor de structs *Aluno* para armazenar os dados. As quatro funcionalidades implementadas são:

Funcionalidade	Implementação	Complexidade
Armazenar dados	Vetor de structs <i>Aluno</i> []	O(1) por inserção
Busca por nome	Busca linear com <code>strcmp()</code>	O(n)
Ordenar por média	Bubble Sort no campo <i>media</i>	O(n²)
Melhor desempenho	Último elemento após ordenação crescente	O(1)

A busca por nome usa a função **`strcmp()`** da biblioteca *string.h* para comparar strings. O aluno com melhor desempenho é identificado em O(1) após a ordenação, pois ocupa sempre a última posição do vetor ordenado por média crescente.